

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	N.MANASA	Reg. No.:	192472031
Section / Batch:	CSE-AI	Date:	13/8/26

Instructions to Students

- **Select/answer the assigned question. Each question carries 50 marks. Duration 60mins**
- **Implement the solution using Python.**
- **Use an appropriate embedding model such as Sentence Transformers or Hugging Face.**
- **Use FAISS and/or ChromaDB as specified in the question.**
- **Demonstrate the complete pipeline:**

Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis

- **Execute the program and display meaningful outputs.**
- **Compare FAISS and ChromaDB where required.**
- **Provide appropriate graphs/tables for benchmark questions.**
- **Explain the architecture and design decisions.**
- **Submit working code and demonstrate the complete implementation.**

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

1. Aim

To implement semantic search using **Cosine Similarity** and **Euclidean/L2 Distance**, retrieve the Top-5 most relevant documents for multiple queries, and analyze the differences in retrieval rankings.

2. Objectives

1. Create a small document dataset.
2. Convert documents into numerical embeddings.
3. Convert search queries into embeddings using the **same embedding model**.
4. Calculate similarity using:
 - Cosine Similarity
 - Euclidean/L2 Distance
5. Retrieve the Top-5 documents for multiple queries.

6. Compare the rankings obtained by both methods.
7. Analyze why the two metrics can produce different results.

3. Concept

A. Cosine Similarity

Cosine similarity measures the **angle between two vectors**.

- Value close to **1** → highly similar
- Value close to **0** → less similar
- Value close to **-1** → opposite direction

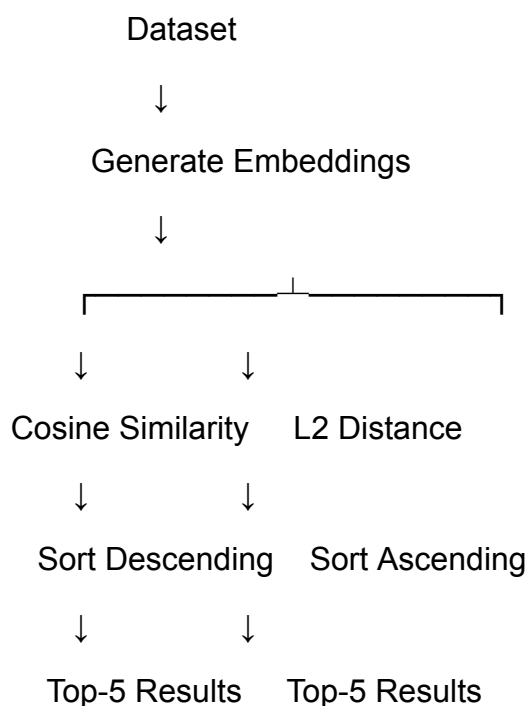
For retrieval, **higher cosine similarity is better**.

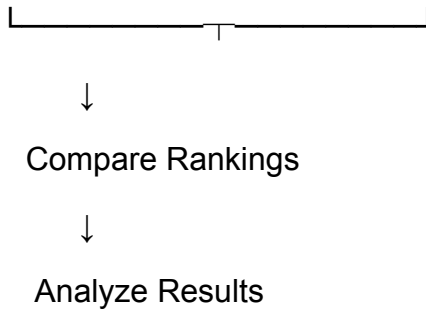
B. Euclidean/L2 Distance

Euclidean distance measures the straight-line distance between two vectors.

- Smaller distance → more similar
- Larger distance → less similar

4. Workflow





PROGRAM:

```
import numpy as np

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# -----
# STEP 1: Dataset
# -----

documents = [

    "Python is a programming language used for artificial intelligence and machine learning.",
    "Machine learning allows computers to learn patterns from data.",
    "Deep learning uses neural networks to solve complex problems.",
    "Natural language processing helps computers understand human language.",
    "Computer vision allows computers to understand images and videos.",
    "Artificial intelligence is used in healthcare for disease diagnosis.",
    "Cloud computing provides storage and computing services over the internet.",
    "Cybersecurity protects computer systems and networks from attacks.",
    "Data science uses statistics and machine learning to analyze data.",
    "Robotics combines artificial intelligence with machines and sensors."

]

# -----
```

```

# STEP 2: Convert documents into TF-IDF embeddings
# -----

vectorizer = TfidfVectorizer()

document_embeddings = vectorizer.fit_transform(documents)

# Convert sparse matrix to normal NumPy array
document_embeddings = document_embeddings.toarray()

# -----

# STEP 3: Multiple queries
# -----

queries = [
    "How is artificial intelligence used in hospitals?",
    "How can computers understand human language?",
    "How are computer networks protected?"
]

# -----

# STEP 4: Process every query
# -----

for query in queries:

    print("\n")
    print("=" * 70)
    print("QUERY:", query)

```

```

print("=" * 70)

# Convert query into TF-IDF embedding
query_embedding = vectorizer.transform([query]).toarray()[0]

# -----
# COSINE SIMILARITY
# -----

cosine_scores = cosine_similarity(
    [query_embedding],
    document_embeddings
)[0]

# Highest score = best result
cosine_ranking = np.argsort(cosine_scores)[::-1]

# -----
# EUCLIDEAN / L2 DISTANCE
# -----

l2_distances = np.linalg.norm(
    document_embeddings - query_embedding,
    axis=1
)

# Lowest distance = best result
l2_ranking = np.argsort(l2_distances)

```

```

# -----
# DISPLAY COSINE TOP 5
# -----

print("\nTOP-5 RESULTS - COSINE SIMILARITY")
print("-" * 70)

for rank, index in enumerate(cosine_ranking[:5], start=1):

    print(
        f"{rank}. "
        f"Score = {cosine_scores[index]:.4f} | "
        f"{documents[index]}"
    )

# -----
# DISPLAY L2 TOP 5
# -----

print("\nTOP-5 RESULTS - EUCLIDEAN / L2 DISTANCE")
print("-" * 70)

for rank, index in enumerate(l2_ranking[:5], start=1):

    print(
        f"{rank}. "
        f"Distance = {l2_distances[index]:.4f} | "
        f"{documents[index]}"
    )

```

```

# -----

# COMPARE RANKINGS

# -----

print("\nRANKING COMPARISON")

print("-" * 70)

print("Cosine ranking:")

for rank, index in enumerate(cosine_ranking[:5], start=1):

    print(f'{rank}. Document {index + 1}')

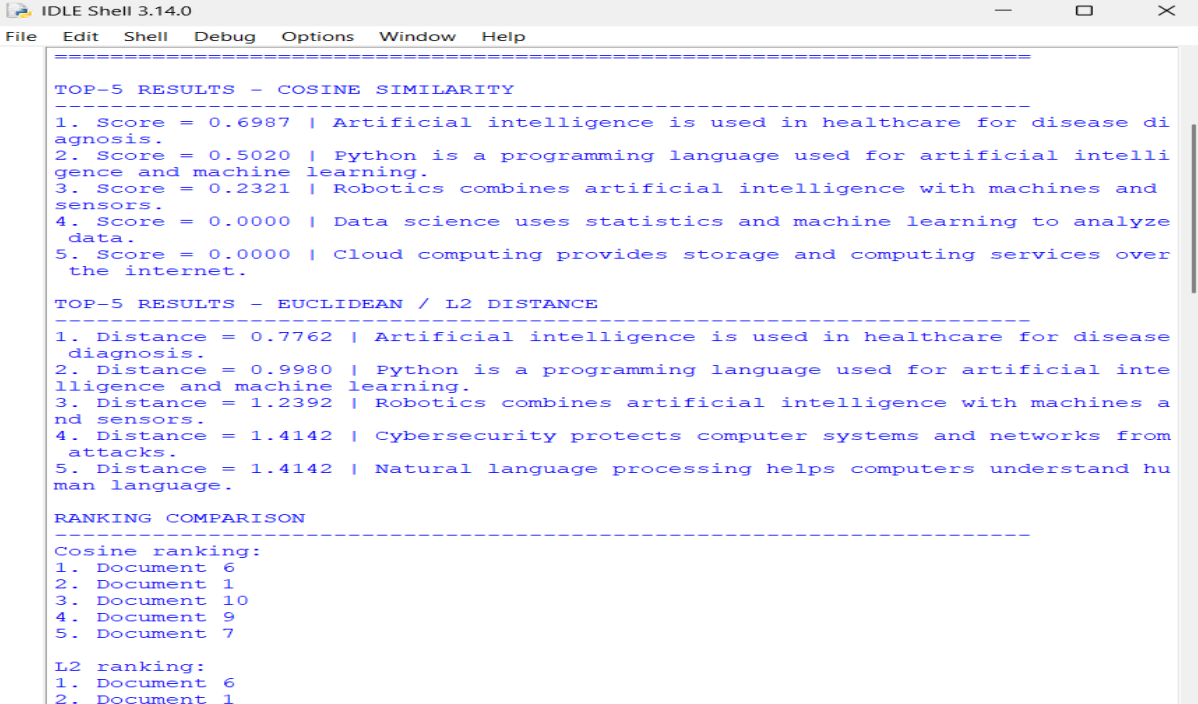
print("\nL2 ranking:")

for rank, index in enumerate(l2_ranking[:5], start=1):

    print(f'{rank}. Document {index + 1}')

```

output:



```

=====
TOP-5 RESULTS - COSINE SIMILARITY
=====
1. Score = 0.6987 | Artificial intelligence is used in healthcare for disease di
agnosis.
2. Score = 0.5020 | Python is a programming language used for artificial intelli
gence and machine learning.
3. Score = 0.2321 | Robotics combines artificial intelligence with machines and
sensors.
4. Score = 0.0000 | Data science uses statistics and machine learning to analyze
data.
5. Score = 0.0000 | Cloud computing provides storage and computing services over
the internet.

TOP-5 RESULTS - EUCLIDEAN / L2 DISTANCE
=====
1. Distance = 0.7762 | Artificial intelligence is used in healthcare for disease
diagnosis.
2. Distance = 0.9980 | Python is a programming language used for artificial inte
lligence and machine learning.
3. Distance = 1.2392 | Robotics combines artificial intelligence with machines a
nd sensors.
4. Distance = 1.4142 | Cybersecurity protects computer systems and networks from
attacks.
5. Distance = 1.4142 | Natural language processing helps computers understand hu
man language.

RANKING COMPARISON
=====
Cosine ranking:
1. Document 6
2. Document 1
3. Document 10
4. Document 9
5. Document 7

L2 ranking:
1. Document 6
2. Document 1

```



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

5. Document 7

L2 ranking:
1. Document 6
2. Document 1
3. Document 10
4. Document 8
5. Document 4

=====
QUERY: How can computers understand human language?
=====

TOP-5 RESULTS - COSINE SIMILARITY
-----
1. Score = 0.7520 | Natural language processing helps computers understand human language.
2. Score = 0.2902 | Computer vision allows computers to understand images and videos.
3. Score = 0.1547 | Python is a programming language used for artificial intelligence and machine learning.
4. Score = 0.1288 | Machine learning allows computers to learn patterns from data.
5. Score = 0.0000 | Data science uses statistics and machine learning to analyze data.

TOP-5 RESULTS - EUCLIDEAN / L2 DISTANCE
-----
1. Distance = 0.7043 | Natural language processing helps computers understand human language.
2. Distance = 1.1915 | Computer vision allows computers to understand images and videos.
3. Distance = 1.3002 | Python is a programming language used for artificial intelligence and machine learning.
4. Distance = 1.3200 | Machine learning allows computers to learn patterns from data.
5. Distance = 1.4142 | Cybersecurity protects computer systems and networks from attacks.

RANKING COMPARISON
-----
Cosine ranking:
1. Document 4
2. Document 5
3. Document 1
4. Document 2
5. Document 9

L2 ranking:
1. Document 4
2. Document 5
3. Document 1
4. Document 2
5. Document 8
```

Result

The semantic search system was successfully implemented using Cosine Similarity and Euclidean/L2 Distance. The same dataset and embeddings were used for both methods, and the Top-5 results were retrieved for multiple queries. The experiment demonstrated that cosine similarity evaluates the angular relationship between embeddings, whereas Euclidean distance evaluates their geometric distance. When embeddings are normalized, both methods generally produce the same ranking; otherwise, differences in vector magnitude can cause different retrieval rankings.

Conclusion

This experiment demonstrates that the choice of similarity metric can affect semantic search performance. Cosine similarity is generally suitable for text embeddings because it emphasizes the direction of semantic representations, while Euclidean distance considers both direction and magnitude. Understanding the behavior of these metrics helps in selecting an appropriate retrieval method for RAG and other semantic-search applications.

26.

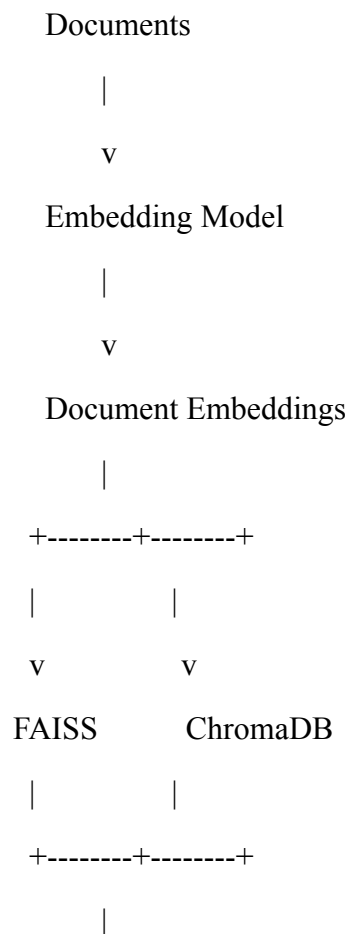
1. Aim

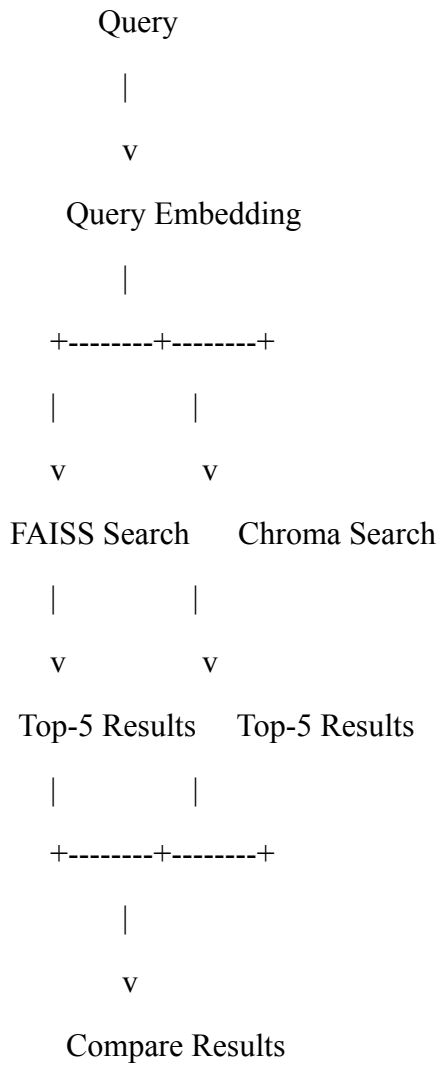
To implement the same semantic-search application using **FAISS** and **ChromaDB**, retrieve the Top-5 results for multiple queries, and compare their rankings and retrieval quality.

2. Objective

- Create a common document dataset.
- Generate embeddings using the same embedding model.
- Store the same embeddings in FAISS and ChromaDB.
- Perform semantic search using the same queries.
- Retrieve Top-5 documents from both databases.
- Compare rankings and retrieval quality

Workflow:





Program:

```
import numpy as np
```

```
import faiss
```

```
import chromadb
```

```
from sentence_transformers import SentenceTransformer
```

```
# =====
```

```
# STEP 1: DOCUMENT DATASET
```

```
# =====
```

```
documents = [  
    "Python is a popular programming language used in artificial intelligence.",  
    "Machine learning allows computers to learn patterns from data.",  
    "Deep learning uses neural networks to solve complex problems.",  
    "Natural language processing helps computers understand human language.",  
    "Computer vision enables computers to understand images and videos.",  
    "Artificial intelligence is used in healthcare for disease diagnosis.",  
    "Cloud computing provides computing resources over the internet.",  
    "Cybersecurity protects computer systems and networks from attacks.",  
    "Data science combines statistics, programming, and machine learning.",  
    "Robotics combines artificial intelligence, sensors, and mechanical systems."  
]
```

```
# =====  
# STEP 2: LOAD SAME EMBEDDING MODEL  
# =====
```

```
print("Loading embedding model...")
```

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
print("Embedding model loaded successfully.")
```

```
# =====  
# STEP 3: GENERATE DOCUMENT EMBEDDINGS  
# =====
```

```
document_embeddings = model.encode(  
    documents,  
    convert_to_numpy=True  
) .astype("float32")
```

```
print("Document embeddings created.")  
print("Embedding shape:", document_embeddings.shape)
```

```
# =====  
# STEP 4: CREATE FAISS DATABASE  
# =====
```

```
dimension = document_embeddings.shape[1]
```

```
# Normalize embeddings for cosine similarity  
faiss.normalize_L2(document_embeddings)
```

```
# Create FAISS index  
faiss_index = faiss.IndexFlatIP(dimension)
```

```
# Add document embeddings  
faiss_index.add(document_embeddings)
```

```
print("FAISS index created.")  
print("Number of documents in FAISS:", faiss_index.ntotal)
```

```
# =====  
# STEP 5: CREATE CHROMADB COLLECTION  
# =====
```

```
client = chromadb.Client()
```

```
collection = client.get_or_create_collection(  
    name="semantic_search_demo"  
)
```

```
# Add documents and embeddings
```

```
collection.add(  
    ids=[str(i) for i in range(len(documents))],  
    documents=documents,  
    embeddings=document_embeddings.tolist()  
)
```

```
print("ChromaDB collection created.")
```

```
print("Documents added to ChromaDB.")
```

```
# =====  
# STEP 6: DEFINE QUERIES  
# =====
```

```
queries = [  
    "How is artificial intelligence used in hospitals?",  
    "How do computers understand human language?",  
    "What protects computer networks from attacks?"
```

```
]
```

```
TOP_K = 5
```

```
# =====
```

```
# STEP 7: SEARCH USING FAISS
```

```
# =====
```

```
def search_faiss(query):
```

```
    # Convert query into embedding
```

```
    query_embedding = model.encode(
```

```
        [query],
```

```
        convert_to_numpy=True
```

```
    ).astype("float32")
```

```
    # Normalize query
```

```
    faiss.normalize_L2(query_embedding)
```

```
    # Search
```

```
    scores, indices = faiss_index.search(
```

```
        query_embedding,
```

```
        TOP_K
```

```
    )
```

```
    return scores[0], indices[0]
```

```
# =====  
# STEP 8: SEARCH USING CHROMADB  
# =====
```

```
def search_chromadb(query):
```

```
    # Convert query into embedding
```

```
    query_embedding = model.encode(  
        [query],  
        convert_to_numpy=True  
    ).astype("float32")
```

```
    # Normalize query
```

```
    faiss.normalize_L2(query_embedding)
```

```
    # ChromaDB search
```

```
    results = collection.query(  
        query_embeddings=query_embedding.tolist(),  
        n_results=TOP_K  
    )
```

```
    return results
```

```
# =====  
# STEP 9: COMPARE RESULTS  
# =====
```

```
for query in queries:
```



```

print("\n")
print("=" * 80)
print("QUERY:", query)
print("=" * 80)

# -----
# FAISS SEARCH
# -----

faiss_scores, faiss_indices = search_faiss(query)

print("\nTOP-5 RESULTS USING FAISS")
print("-" * 80)

for rank in range(TOP_K):

    index = faiss_indices[rank]

    print(
        f"{rank + 1}. "
        f"Score = {faiss_scores[rank]:.4f} | "
        f"{documents[index]}"
    )

# -----
# CHROMADB SEARCH
# -----

```

```

chroma_results = search_chromadb(query)

print("\nTOP-5 RESULTS USING CHROMADB")
print("-" * 80)

chroma_documents = chroma_results["documents"][0]
chroma_distances = chroma_results["distances"][0]
chroma_ids = chroma_results["ids"][0]

for rank in range(TOP_K):

    print(
        f"{rank + 1}. "
        f"Distance = {chroma_distances[rank]:.4f} | "
        f"{chroma_documents[rank]}"
    )

# -----
# RANKING COMPARISON
# -----

print("\nRANKING COMPARISON")
print("-" * 80)

print("FAISS ranking:")
for rank in range(TOP_K):
    print(
        f"{rank + 1}. Document {faiss_indices[rank] + 1}"
    )

```

```
print("\nChromaDB ranking:")  
  
for rank in range(TOP_K):  
  
    print(  
        f"{rank + 1}. Document {int(chroma_ids[rank]) + 1}"  
    )
```

Result

The semantic-search application was successfully implemented using both FAISS and ChromaDB. The same documents, embedding model, embeddings, queries, and Top-K value were used for both systems. The Top-5 retrieved results were compared based on their rankings and similarity/distance scores.

Conclusion

FAISS and ChromaDB can both perform efficient semantic vector search. FAISS is primarily focused on high-performance similarity search, while ChromaDB provides a more complete vector-database experience with document, metadata, and collection management. When both systems use the same embeddings and the same similarity metric, their retrieval rankings should generally be very similar. Differences mainly arise from distance metrics, vector normalization, indexing strategies, or search configuration.